Pretty-print ☑


```
{
  "openapi": "3.0.3",
  "info": {
    "title": "Event Scanner API",
    "version": "1.0.0",
    "description": "Machine-to-machine API for check-in scanner apps. Authenticate every request
with a bearer API key issued by the event organizer:\n\n`Authorization: Bearer evk_...`\n\nKeys are
global — the target event is named per request by its **slug**. The slug is the event's participant
landing-page URL segment (the organizer shares it from the admin console); e.g. for
`https://portal.example/spring-gala` the slug is `spring-gala`.\n\nA key can be rotated or revoked
at any time, which takes effect immediately."
  },
  "servers": [
    {
      "url": "/api/scanner/v1",
      "description": "Scanner API v1"
    }
  ],
  "security": [
    {
      "bearerAuth": []
    }
  ],
  "tags": [
    {
      "name": "Scanner",
      "description": "Sub-event listing and check-in"
    }
  ],
  "paths": {
    "/events/{slug}/sub-events": {
      "get": {
        "tags": [
          "Scanner"
        ],
        "summary": "List an event's sub-events",
        "description": "Returns the sub-events of the event with this slug, ordered for display. Use
the returned `id` as `subEventId` when checking in.",
        "parameters": [
          {
            "name": "slug",
            "in": "path",
            "required": true,
            "schema": {
              "type": "string"
            },
            "description": "Event slug (participant landing-page URL segment), from the admin
console. e.g. `spring-gala`."
          }
        ],
        "responses": {
          "200": {
            "description": "Sub-events for the event.",
            "content": {
              "application/json": {
                "schema": {
                  "type": "object",
                  "additionalProperties": false,
                  "required": [
                    "data"
                  ],
                  "properties": {
                    "data": {
                      "type": "array",
                      "items": {
                        "$ref": "#/components/schemas/SubEvent"
                      }
```

Pretty-print


```
                  }
                }
              }
            }
          },
          "401": {
            "$ref": "#/components/responses/Unauthorized"
          },
          "404": {
            "$ref": "#/components/responses/EventNotFound"
          }
        }
      }
    },
    "/events/{slug}/participants": {
      "get": {
        "tags": [
          "Scanner"
        ],
        "summary": "List / search an event's participants",
        "description": "Participant directory for **name-based check-in** — the fallback when a
participant cannot show their QR code (flat battery, lost code). Search by name or printed code with
`q`, then pass the chosen `id` to `POST /checkin` as `participantId`.\n\nResults are paginated and
ordered by name. Rate limited to 30 requests per minute per key.\n\nThe response excludes email and
phone. It includes `uniqueCode` so an operator can tell same-name participants apart; since that
code is also the participant's Portal login credential, the API key that reads this list is
credential-grade — hold it tightly and revoke it on any leak.",
        "parameters": [
          {
            "name": "slug",
            "in": "path",
            "required": true,
            "schema": {
              "type": "string"
            },
            "description": "Event slug (participant landing-page URL segment), from the admin
console. e.g. `spring-gala`."
          },
          {
            "name": "q",
            "in": "query",
            "required": false,
            "schema": {
              "type": "string",
              "maxLength": 100
            },
            "description": "Case-insensitive substring match on participant name or unique code.
Omit to list everyone.",
            "example": "budi"
          },
          {
            "name": "subEventId",
            "in": "query",
            "required": false,
            "schema": {
              "type": "string"
            },
            "description": "Scope `checkedIn` / `totalScans` / `firstScanAt` to this sub-event. Omit
for event-wide totals. Does not filter `seats`, which always lists every sub-event the participant
has a seat in."
          },
          {
            "name": "limit",
            "in": "query",
            "required": false,
            "schema": {
              "type": "integer",
              "minimum": 1,
```

Pretty-print


```
            "default": 50
          }
        },
        {
          "name": "offset",
          "in": "query",
          "required": false,
          "schema": {
            "type": "integer",
            "minimum": 0,
            "default": 0
          }
        }
      ],
      "responses": {
        "200": {
          "description": "A page of participants.",
          "content": {
            "application/json": {
              "schema": {
                "type": "object",
                "additionalProperties": false,
                "required": [
                  "data"
                ],
                "properties": {
                  "data": {
                    "$ref": "#/components/schemas/ParticipantPage"
                  }
                }
              }
            }
          }
        },
        "401": {
          "$ref": "#/components/responses/Unauthorized"
        },
        "404": {
          "$ref": "#/components/responses/EventNotFound"
        },
        "422": {
          "$ref": "#/components/responses/ValidationError"
        },
        "429": {
          "$ref": "#/components/responses/RateLimited"
        }
      }
    }
  },
  "/checkin": {
    "post": {
      "tags": [
        "Scanner"
      ],
      "summary": "Check a participant in",
      "description": "Records a check-in, identifying the participant **either** by their scanned `code` **or** by the `participantId` returned from `GET /events/{slug}/participants`. Supply exactly one of the two.\n\nFor a single-entry sub-event, a repeat scan returns `alreadyCheckedIn: true` and writes no new record. Rate limited to 60 requests per minute per key.",
      "requestBody": {
        "required": true,
        "content": {
          "application/json": {
            "schema": {
              "$ref": "#/components/schemas/CheckinRequest"
            }
          }
        }
```

Pretty-print


```
        "responses": {
          "200": {
            "description": "Check-in result.",
            "content": {
              "application/json": {
                "schema": {
                  "type": "object",
                  "additionalProperties": false,
                  "required": [
                    "data"
                  ],
                  "properties": {
                    "data": {
                      "$ref": "#/components/schemas/CheckinResult"
                    }
                  }
                }
              }
            }
          },
          "401": {
            "$ref": "#/components/responses/Unauthorized"
          },
          "404": {
            "description": "The code, the participant, or the named sub-event was not found.",
            "content": {
              "application/json": {
                "schema": {
                  "$ref": "#/components/schemas/Error"
                },
                "examples": {
                  "codeNotFound": {
                    "summary": "Unknown participant code",
                    "value": {
                      "error": {
                        "code": "CODE_NOT_FOUND",
                        "message": "Code not found"
                      }
                    }
                  },
                  "participantNotFound": {
                    "summary": "participantId unknown, or not part of this event (no 409 — the API
does not confirm the id exists elsewhere)",
                    "value": {
                      "error": {
                        "code": "NOT_FOUND",
                        "message": "Participant not found"
                      }
                    }
                  },
                  "subEventNotFound": {
                    "summary": "Sub-event not part of this event",
                    "value": {
                      "error": {
                        "code": "NOT_FOUND",
                        "message": "Sub-event not found"
                      }
                    }
                  }
                }
              }
            }
          },
          "409": {
            "description": "The code belongs to a different event than the one supplied. Code path
only — a `participantId` from another event returns 404.",
            "content": {
              "application/json": {
```

Pretty-print


```
                  "$ref": "#/components/schemas/Error"
                },
                "example": {
                  "error": {
                    "code": "WRONG_EVENT",
                    "message": "This code belongs to a different event"
                  }
                }
              }
            }
          },
          "422": {
            "$ref": "#/components/responses/ValidationError"
          },
          "429": {
            "$ref": "#/components/responses/RateLimited"
          }
        }
      }
    }
  },
  "components": {
    "securitySchemes": {
      "bearerAuth": {
        "type": "http",
        "scheme": "bearer",
        "bearerFormat": "evk_...",
        "description": "API key issued by the organizer. Send as `Authorization: Bearer evk_...`."
      }
    },
    "schemas": {
      "SubEvent": {
        "type": "object",
        "additionalProperties": false,
        "required": [
          "id",
          "name",
          "description",
          "entryType",
          "seatingEnabled",
          "sortOrder"
        ],
        "properties": {
          "id": {
            "type": "string"
          },
          "name": {
            "type": "string"
          },
          "description": {
            "type": "string"
          },
          "entryType": {
            "type": "string",
            "enum": [
              "single",
              "multiple"
            ],
            "description": "`single`: one check-in per participant (repeat scans are idempotent).
`multiple`: every scan is recorded."
          },
          "seatingEnabled": {
            "type": "boolean"
          },
          "sortOrder": {
            "type": "integer"
          }
        }
```

Pretty-print


```
      "ParticipantPage": {
        "type": "object",
        "additionalProperties": false,
        "required": [
          "total",
          "limit",
          "offset",
          "participants"
        ],
        "properties": {
          "total": {
            "type": "integer",
            "minimum": 0,
            "description": "Total participants matching the search, ignoring pagination."
          },
          "limit": {
            "type": "integer",
            "minimum": 1,
            "maximum": 200
          },
          "offset": {
            "type": "integer",
            "minimum": 0
          },
          "participants": {
            "type": "array",
            "items": {
              "$ref": "#/components/schemas/Participant"
            }
          }
        }
      },
      "Participant": {
        "type": "object",
        "additionalProperties": false,
        "required": [
          "id",
          "fullName",
          "uniqueCode",
          "affiliation",
          "jobTitle",
          "participantType",
          "rsvpStatus",
          "seats",
          "checkedIn",
          "totalScans",
          "firstScanAt"
        ],
        "properties": {
          "id": {
            "type": "string",
            "description": "Opaque participant id. Pass as `participantId` to `POST /checkin`."
          },
          "fullName": {
            "type": "string"
          },
          "uniqueCode": {
            "type": "string",
            "description": "The participant's unique code (e.g. `PEG-D-7TGY`), for disambiguating
same-name participants against a printed badge. This is also the participant's Portal login
credential, so treat the API key that reads this list as credential-grade.",
            "example": "PEG-D-7TGY"
          },
          "affiliation": {
            "type": "string",
            "nullable": true
          },
          "jobTitle": {
```

Pretty-print


```
            "nullable": true
          },
          "participantType": {
            "type": "string",
            "enum": [
              "Committee",
              "Delegates",
              "Media",
              "Speaker",
              "Internal",
              "Guest"
            ],
            "description": "Participant classification, as a display label. `Internal` and `Guest`
are legacy classifications carried by participants created before the Committee/Delegates rename.",
            "example": "Delegates"
          },
          "rsvpStatus": {
            "type": "string",
            "enum": [
              "invited",
              "confirmed"
            ]
          },
          "seats": {
            "type": "array",
            "description": "Seat assignments, one entry per sub-event the participant has a seat in,
in sub-event display order. Empty when no seats are assigned.",
            "items": {
              "type": "object",
              "additionalProperties": false,
              "required": [
                "subEventId",
                "subEventName",
                "label"
              ],
              "properties": {
                "subEventId": {
                  "type": "string"
                },
                "subEventName": {
                  "type": "string"
                },
                "label": {
                  "type": "string",
                  "example": "Table 5"
                }
              }
            }
          },
          "checkedIn": {
            "type": "boolean",
            "description": "Whether this participant has any check-in in the requested scope."
          },
          "totalScans": {
            "type": "integer",
            "minimum": 0,
            "description": "Check-ins recorded in the requested scope."
          },
          "firstScanAt": {
            "type": "string",
            "format": "date-time",
            "nullable": true,
            "description": "First check-in in scope; null when never checked in."
          }
        }
      },
      "CheckinRequest": {
        "type": "object",
```

Pretty-print


```
        "required": [
          "slug"
        ],
        "description": "Supply exactly one of `code` (QR scan) or `participantId` (name
selection).",
        "oneOf": [
          {
            "required": [
              "code"
            ]
          },
          {
            "required": [
              "participantId"
            ]
          }
        ],
        "properties": {
          "code": {
            "type": "string",
            "minLength": 4,
            "maxLength": 32,
            "description": "Scanned participant code. Dashes and case are normalized server-side.
Mutually exclusive with `participantId`.",
            "example": "PEG-G-7TGY"
          },
          "participantId": {
            "type": "string",
            "minLength": 1,
            "description": "Participant `id` from `GET /events/{slug}/participants`, for checking in
someone who cannot show a QR code. Mutually exclusive with `code`; records the check-in as
`manual`."
          },
          "slug": {
            "type": "string",
            "minLength": 1,
            "description": "Event slug (participant landing-page URL segment), from the admin
console.",
            "example": "spring-gala"
          },
          "subEventId": {
            "type": "string",
            "minLength": 1,
            "description": "Optional sub-event to scope the check-in to. Omit for an event-level
check-in."
          }
        }
      },
      "CheckinResult": {
        "type": "object",
        "additionalProperties": false,
        "required": [
          "status",
          "checkinMethod",
          "entryType",
          "participantName",
          "affiliation",
          "subEventSeats",
          "seatDisplay",
          "eventName",
          "subEventName",
          "scannedAt",
          "totalScans",
          "reentry",
          "alreadyCheckedIn"
        ],
        "properties": {
          "status": {
```

Pretty-print


```
            "enum": [
              "checked_in",
              "re_entry",
              "already_checked_in"
            ],
            "description": "Switch on this. `checked_in`: recorded, first time in scope (show OK).
`re_entry`: recorded again on a multiple-entry sub-event (show OK). `already_checked_in`: single-
entry duplicate — nothing recorded (show a warning)."
          },
          "checkinMethod": {
            "type": "string",
            "enum": [
              "qr",
              "manual"
            ],
            "description": "How the participant was identified: `qr` (a `code` was scanned) or
`manual` (an operator selected them by name). Recorded in the organizer's audit log."
          },
          "entryType": {
            "type": "string",
            "enum": [
              "single",
              "multiple"
            ],
            "nullable": true,
            "description": "Entry type of the scanned sub-event; null for an event-level check-in."
          },
          "participantName": {
            "type": "string"
          },
          "affiliation": {
            "type": "string",
            "nullable": true
          },
          "subEventSeats": {
            "type": "object",
            "additionalProperties": {
              "type": "string"
            },
            "description": "Map of subEventId → seat label for this participant."
          },
          "seatDisplay": {
            "type": "string",
            "description": "Human-readable seat(s) for the scanned scope."
          },
          "eventName": {
            "type": "string"
          },
          "subEventName": {
            "type": "string",
            "nullable": true
          },
          "scannedAt": {
            "type": "string",
            "format": "date-time"
          },
          "totalScans": {
            "type": "integer",
            "minimum": 1,
            "description": "Total check-ins recorded in the scanned scope."
          },
          "reentry": {
            "type": "boolean",
            "description": "Legacy — prefer `status`. True when this is not the first scan in
scope."
          },
          "alreadyCheckedIn": {
            "type": "boolean",
```

Pretty-print

```
checked in (no new record)."
          }
        }
      },
      "Error": {
        "type": "object",
        "additionalProperties": false,
        "required": [
          "error"
        ],
        "properties": {
          "error": {
            "type": "object",
            "additionalProperties": false,
            "required": [
              "code",
              "message"
            ],
            "properties": {
              "code": {
                "type": "string"
              },
              "message": {
                "type": "string"
              },
              "details": {
                "type": "object",
                "additionalProperties": {
                  "type": "array",
                  "items": {
                    "type": "string"
                  }
                },
                "nullable": true
              }
            }
          }
        }
      }
    },
    "responses": {
      "Unauthorized": {
        "description": "Missing, malformed, revoked, or expired API key.",
        "content": {
          "application/json": {
            "schema": {
              "$ref": "#/components/schemas/Error"
            },
            "example": {
              "error": {
                "code": "UNAUTHENTICATED",
                "message": "Invalid or missing API key"
              }
            }
          }
        }
      },
      "EventNotFound": {
        "description": "No event with that slug.",
        "content": {
          "application/json": {
            "schema": {
              "$ref": "#/components/schemas/Error"
            },
            "example": {
              "error": {
                "code": "NOT_FOUND",
                "message": "Event not found"
```

Pretty-print


```
            }
          }
        }
      },
      "ValidationError": {
        "description": "The request body failed validation.",
        "content": {
          "application/json": {
            "schema": {
              "$ref": "#/components/schemas/Error"
            },
            "example": {
              "error": {
                "code": "VALIDATION",
                "message": "Invalid request",
                "details": {
                  "code": [
                    "Required"
                  ]
                }
              }
            }
          }
        }
      },
      "RateLimited": {
        "description": "Too many requests (60/min per key). See the `Retry-After` header.",
        "headers": {
          "Retry-After": {
            "description": "Seconds until the key can make another check-in request.",
            "schema": {
              "type": "integer",
              "minimum": 1
            }
          }
        },
        "content": {
          "application/json": {
            "schema": {
              "$ref": "#/components/schemas/Error"
            },
            "example": {
              "error": {
                "code": "RATE_LIMITED",
                "message": "Too many requests"
              }
            }
          }
        }
      }
    }
  }
}
```